

Wedding Digital Invitation

Deployment Guide

Complete guide for deploying the wedding invitation platform on a fresh server instance from GitHub.

Technology Stack

- Frontend** : Next.js 14 + Tailwind CSS + Swiper
- Backend** : Node.js + Express.js
- Database** : PostgreSQL 16
- Cache** : Redis 7
- Proxy** : Nginx 1.27
- Containerization** : Docker + Docker Compose

Version 1.0 | June 2026
Generated by Hermes Agent

Table of Contents

- 1.** Why "Missing" Files Aren't Actually Missing
- 2.** Architecture Overview
- 3.** Prerequisites (Fresh Instance)
- 4.** Step-by-Step Deployment
- 5.** Post-Deployment Setup
- 6.** Common Operations
- 7.** Environment Variables Reference
- 8.** Database Migrations
- 9.** Troubleshooting
- 10.** Production Checklist

1. Why "Missing" Files Aren't Actually Missing

When you push this project to GitHub, several large or sensitive items are intentionally excluded via `.gitignore`. Here is how they get recreated on a new instance:

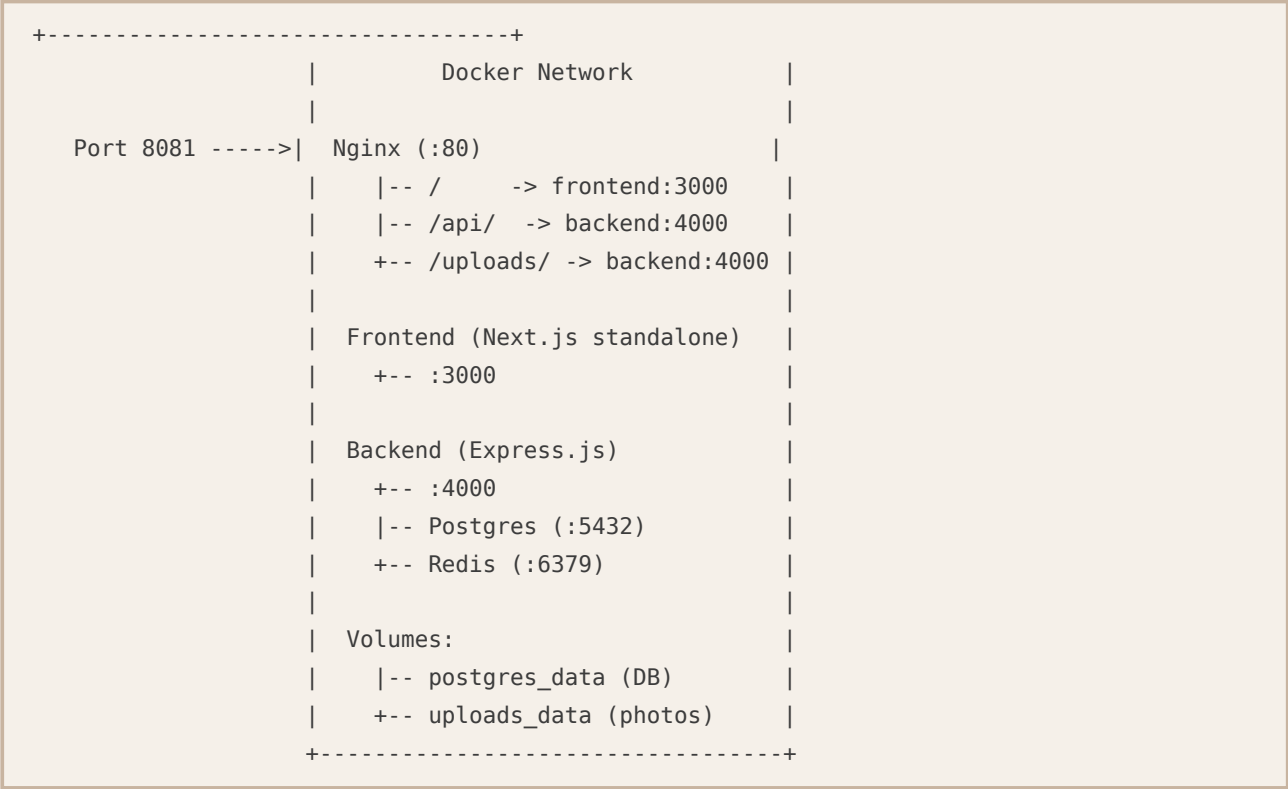
Excluded File/Folder	Size	How It's Recreated
frontend/node_modules/	~360 MB	Docker rebuild - npm install
backend/node_modules/	~5 MB	Docker rebuild - npm install --production
frontend/.next/	~145 MB	Docker rebuild - npm run build (standalone)
.env	< 1 KB	Manual - copy <code>.env.example</code> and fill values
uploads/	varies	Docker volume - empty on first run
postgres_data/	varies	Docker volume - migrations auto-run

The Key Insight

You never need `node_modules` or `.next` on the host machine. Docker builds them inside isolated containers and discards the intermediate layers. The final images are small (~150 MB frontend, ~50 MB backend).

```
GitHub repo (source code only)
|
v
docker compose up --build
|
|---> Frontend Dockerfile:
|   1. npm install      <- recreates node_modules (inside container)
|   2. npm run build    <- recreates .next/ (inside container)
|   3. Copies only standalone output to final image
|
|---> Backend Dockerfile:
|   1. npm install --production <- recreates node_modules (inside container)
|   2. Copies src/ to final image
|
|---> PostgreSQL container:
|   1. Starts fresh with empty DB
|   2. Runs 001_init.sql on first boot
|   3. Backend also runs migrations on startup (idempotent)
|
+---> Redis container:
|   1. Starts fresh with empty cache
```

2. Architecture Overview



Services

Service	Container	Port	Purpose
nginx	wedding_nginx	8081->80	Reverse proxy
frontend	wedding_frontend	3000 (int)	Next.js standalone server
backend	wedding_backend	4000->4000	Express.js API server
postgres	wedding_postgres	5432->5432	PostgreSQL 16 database
redis	wedding_redis	6379->6379	Redis 7 cache

3. Prerequisites (Fresh Instance)

3.1 Install Docker + Docker Compose

```
# Ubuntu / Debian
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
# Log out and back in for group change to take effect

# Verify
docker --version          # Docker version 24+
docker compose version    # Docker Compose v2+
```

3.2 Install Git

```
sudo apt-get update && sudo apt-get install -y git
```

3.3 System Requirements

- > RAM: Minimum 2 GB (4 GB recommended for Next.js build)
- > Disk: 5 GB free (Docker images + volumes)
- > CPU: 1 vCPU minimum (2+ recommended for build speed)

4. Step-by-Step Deployment

Step 1 - Clone the Repository

```
cd ~
git clone https://github.com/YOUR_USERNAME/wedding-invitation.git
cd wedding-invitation
```

Step 2 - Create the Environment File

```
# Copy the template
cp .env.example .env

# Edit with real values
nano .env
```

Critical values to change:

- JWT_SECRET: Generate with 'openssl rand -base64 32'
- DB_PASSWORD: Change from default 'wedding_secret'
- BASE_URL / FRONTEND_URL: Update if using a real domain
- NEXT_PUBLIC_API_URL: Leave empty - Nginx handles routing
- NEXT_PUBLIC_GOOGLE_MAPS_KEY: Optional, for ceremony location map

Never commit .env to Git. It is in .gitignore for security.

Step 3 - Build and Start

```
# Build all images and start containers
docker compose up -d --build
```

What happens (takes 3-10 minutes depending on server):

1. Docker pulls node:20-alpine, postgres:16-alpine, redis:7-alpine, nginx:1.27-alpine
2. Frontend image: npm install -> npm run build -> copies standalone output
3. Backend image: npm install --production -> copies src/
4. PostgreSQL starts -> runs 001_init.sql migration (creates tables)
5. Redis starts
6. Backend starts -> runs migrations (idempotent) -> listens on :4000
7. Frontend starts -> serves Next.js standalone on :3000
8. Nginx starts -> proxies traffic

Step 4 - Verify

```
# Check all containers are running and healthy
docker compose ps

# Test the endpoints
curl http://localhost:8081/           # Frontend (should return HTML)
curl http://localhost:8081/health    # Backend health (should return JSON)
curl http://localhost:8081/api/invitations  # API (should return JSON)
```

Open in browser: http://YOUR_SERVER_IP:8081

Step 5 - Configure Firewall (Optional)

```
# Open port 8081
sudo ufw allow 8081/tcp

# Or if using a custom domain with SSL (port 80/443):
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

5. Post-Deployment Setup

5.1 Create Admin Account

The first user registration becomes the admin. Register at:

```
http://YOUR_SERVER_IP:8081/register
```

5.2 Create Your First Invitation

1. Log in at /login
2. Go to Dashboard -> "Create New Invitation"
3. Fill in couple names, event details, venue, etc.
4. Upload bride/groom photos and gallery images
5. Invitation available at `http://YOUR_SERVER_IP:8081/your-slug`

5.3 Upload Existing Photos (If Migrating)

If you have photos from a previous deployment:

```
# Copy photos into the backend container's upload volume
docker cp ./uploads/. wedding_backend:/uploads/

# Or mount the volume and copy directly
docker run --rm -v wedding-invitation_uploads_data:/data \
  -v $(pwd)/uploads:/src alpine cp -a /src/. /data/
```


6. Common Operations

Rebuild After Code Changes

```
# Rebuild only the service that changed
docker compose up -d --build frontend

# Rebuild everything
docker compose up -d --build
```

View Logs

```
# All services
docker compose logs -f

# Specific service (last 50 lines, follow)
docker compose logs -f --tail 50 frontend
docker compose logs -f --tail 50 backend
docker compose logs -f --tail 50 nginx
```

Restart Nginx (Fixes 502 After Container Recreate)

When you recreate the frontend or backend container, their internal Docker IPs change. Nginx caches the old IP, causing 502 Bad Gateway. Fix: `docker compose restart nginx`

```
docker compose restart nginx
```

Stop Everything

```
# Containers stop but volumes (DB + uploads) are preserved
docker compose down
```

Wipe Everything (Fresh Start)

This deletes the database AND all uploaded photos. Irreversible!

```
docker compose down -v
```

Backup Database

```
# Export database to file
docker exec wedding_postgres pg_dump -U wedding wedding > backup_$(date +%Y%m%d).sql

# Restore from file
cat backup_20260101.sql | docker exec -i wedding_postgres psql -U wedding wedding
```

Backup Uploaded Photos

```
docker run --rm -v wedding-invitation_uploads_data:/data \
-v $(pwd):/backup alpine \
tar czf /backup/uploads_backup_$(date +%Y%m%d).tar.gz -C /data .
```

7. Environment Variables Reference

Variable	Service	Default	Description
NEXT_PUBLIC_API_URL	Frontend	"" (empty)	Backend API URL (empty=Ngix proxy)
NEXT_PUBLIC_GOOGLE_MAPS_KEY	Frontend	-	Google Maps Embed API key
PORT	Backend	4000	Backend listen port
NODE_ENV	Backend	production	Node environment
DB_HOST	Backend	postgres	PostgreSQL hostname (Docker svc)
DB_PORT	Backend	5432	PostgreSQL port
DB_NAME	Backend	wedding	Database name
DB_USER	Backend	wedding	Database user
DB_PASSWORD	Backend	wedding_secret	DB password (must match compose)
REDIS_URL	Backend	redis://redis:6379	Redis connection string
JWT_SECRET	Backend	-	Secret for JWT (generate random)
BASE_URL	Backend	localhost:4000	Backend URL for email links
FRONTEND_URL	Backend	localhost:3000	Frontend URL for CORS
UPLOAD_DIR	Backend	/uploads	Photo upload dir (in container)

8. Database Migrations

Migrations are SQL files in `backend/migrations/`. They run automatically in two places:

1. PostgreSQL container first boot - `docker-compose.yml` mounts `001_init.sql` to `/docker-entrypoint-initdb.d/`, which PostgreSQL auto-executes on first startup only.
2. Backend startup - `backend/src/index.js` has an `initDb()` function that reads all `.sql` files and runs them. It catches 'already exists' errors, so it is idempotent.

Adding New Migrations

```
# Create a new migration file (numbered sequentially)
touch backend/migrations/003_add_gift_registry.sql

# The backend will pick it up automatically on next restart.
# No manual SQL needed.
```

9. Troubleshooting

502 Bad Gateway

Cause: Nginx can't reach frontend/backend (usually after recreating a container).

```
docker compose restart nginx
```

Frontend Container Won't Start

Cause: Next.js build failed (usually out of memory).

```
# Check logs
docker compose logs frontend

# If OOM: the Dockerfile sets NODE_OPTIONS=--max-old-space-size=3072
# Ensure the server has at least 4 GB RAM, or add swap:
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
```

Database Connection Refused

```
# Check if postgres is healthy
docker compose ps postgres

# If not healthy, check logs
docker compose logs postgres

# If DB password mismatch between .env and docker-compose.yml,
# the postgres container uses the password from docker-compose.yml.
# The backend reads from .env. They MUST match.
```

Port 8081 Already in Use

Edit docker-compose.yml nginx service:

```
nginx:
  ports:
    - "YOUR_PORT:80"    # e.g., "3000:80"
```

10. Production Checklist

- [] .env created with real values (especially JWT_SECRET)
- [] JWT_SECRET is a random 32+ character string (not default)
- [] DB_PASSWORD changed from default wedding_secret
- [] Firewall configured (only expose needed ports)
- [] HTTPS configured (use Nginx + Let's Encrypt or a load balancer)
- [] Database backups scheduled (cron + pg_dump)
- [] Upload backups scheduled
- [] Docker compose auto-restart on reboot (already set via restart: unless-stopped)
- [] Monitor disk space (uploads + postgres volume growth)

End of Deployment Guide - Wedding Digital Invitation v1.0